Lab 1: Classical Encryption Techniques

Java Implementation & Analysis

Computer Security Course

Lab Objectives & Overview

- Understand the fundamental concepts of Cryptography and Cryptanalysis.
- Explore classical substitution techniques: Caesar, Monoalphabetic, and Playfair ciphers.
- Implement these classical algorithms using the Java programming language.
- Analyze the security and limitations of each technique through practical coding.

Introduction to Cryptology

Core Definitions

Cryptology: The study of techniques for ensuring information secrecy and authenticity.

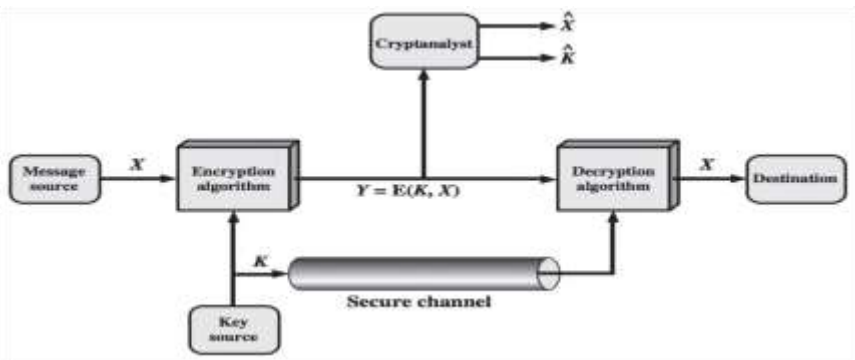
Cryptography: Focuses on the design of secure communication techniques.

Cryptanalysis: Involves defeating techniques to recover original information.

Security Levels

Unconditionally Secure: Mathematically unbreakable, regardless of computing power.

Computationally Secure: Breaking the cipher exceeds the value or lifetime of the information.



Caesar Cipher:
The Shift Technique

Mechanism

The Caesar Cipher is a basic substitution technique where each letter in the plaintext is shifted by a fixed number of positions down the alphabet.

Example: With a shift key of 8, the letter 'C' (index 2) becomes 'K' (index 10).

$C = (P + k) \bmod 26$

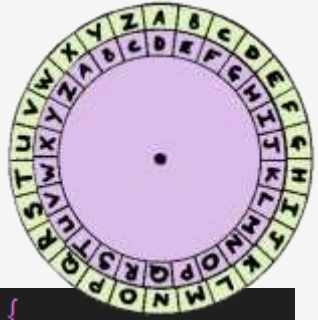
P = Plaintext Index, k = Key, C = Ciphertext Index

Security Analysis

While historically significant, the Caesar cipher offers minimal security in modern contexts.

- Highly vulnerable to brute-force attacks.
- Only 25 possible keys (excluding key 0).
- Easily broken using frequency analysis.

Java Implementation: Caesar Cipher



```java
public class CaesarCipher {

    public static String encrypt(String text, int key) {
        StringBuilder result = new StringBuilder();

        for (char character : text.toCharArray()) {
            if (Character.isLetter(character)) {

                char base = Character.isUpperCase(character) ? 'A' : 'a';
                char encryptedChar =
                        (char) ((character - base + key) % 26 + base);

                result.append(encryptedChar);

            } else {
                result.append(character);
            }
        }

        return result.toString();
    }

    public static String decrypt(String text, int key) {
        // Decryption is encryption with the reverse shift
        return encrypt(text, 26 - (key % 26));
    }
```

```java
    public static void main(String[] args) {
        String plaintext = "Computer";
        int key = 8;

        String ciphertext = encrypt(plaintext, key);
        String decryptedText = decrypt(ciphertext, key);

        System.out.println("Plaintext : " + plaintext);
        System.out.println("Ciphertext: " + ciphertext);
        System.out.println("Decrypted : " + decryptedText);
    }
}
```

Monoalphabetic Cipher: Fixed Mapping

Concept & Mechanism

Unlike Caesar's fixed shift, Monoalphabetic ciphers use a **random permutation** of the alphabet. Each plaintext letter maps to a unique ciphertext letter based on a predefined mapping table.

Key Space

The total number of possible keys is **26!** (approximately 4.03×10^{26}). This massive key space makes brute-force attacks computationally infeasible.

Security Vulnerability

Despite the large key space, the cipher is highly vulnerable to **Frequency Analysis**.

The relative frequency of letters in the plaintext is preserved in the ciphertext, allowing attackers to map common ciphertext letters to common language letters (e.g., 'E', 'T', 'A').

Example Mapping

Plaintext:	a b c d e f g h i j k l m ...
Ciphertext:	D Q P E R S T A F W X G O ...

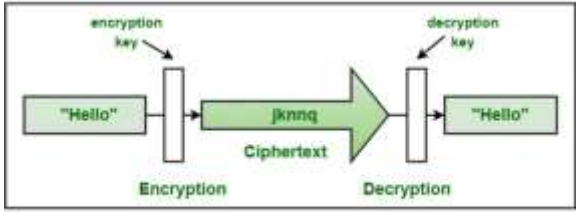
Example: "computer" → "PBOVIMRN"

Java Implementation: Monoalphabetic Cipher (Part 1)

```java
import java.util.HashMap;
import java.util.Map;

public class Monoalphabetic {

    public static String encrypt(String text, String key) {
        String alphabet = "abcdefghijklmnopqrstuvwxyz";
        Map<Character, Character> map = new HashMap<>();

        for (int i = 0; i < 26; i++) {
            map.put(alphabet.charAt(i), key.charAt(i));
        }

        StringBuilder sb = new StringBuilder();

        for (char c : text.toLowerCase().toCharArray()) {
            sb.append(map.getOrDefault(c, c));
        }

        return sb.toString();
    }
```

Implementation Details

The implementation uses a **HashMap** to store the character mapping for O(1) lookup time. The plaintext is converted to lowercase to match the mapping table.



Cryptography

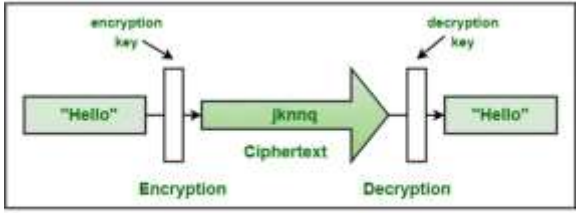
Note: Non-alphabetic characters are preserved using getOrDefault().

Java Implementation: Monoalphabetic Cipher (Part 2)

```java
    public static String decrypt(String text, String key) {
        String alphabet = "abcdefghijklmnopqrstuvwxyz";
        Map<Character, Character> reverseMap = new HashMap<>();

        // Build reverse substitution map
        for (int i = 0; i < 26; i++) {
            reverseMap.put(key.charAt(i), alphabet.charAt(i));
        }

        StringBuilder sb = new StringBuilder();

        for (char c : text.toLowerCase().toCharArray()) {
            sb.append(reverseMap.getOrDefault(c, c));
        }

        return sb.toString();
    }

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        String key = "dqperstafwxgobvhnclizumlyj";
        String plaintext = "computer";

        String ciphertext = encrypt(plaintext, key);
        String decryptedText = decrypt(ciphertext, key);

        System.out.println("Plaintext : " + plaintext);
        System.out.println("Ciphertext: " + ciphertext);
        System.out.println("Decrypted : " + decryptedText);
    }
}
```

Implementation Details

The implementation uses a **HashMap** to store the character mapping for O(1) lookup time. The plaintext is converted to lowercase to match the mapping table.



Cryptography

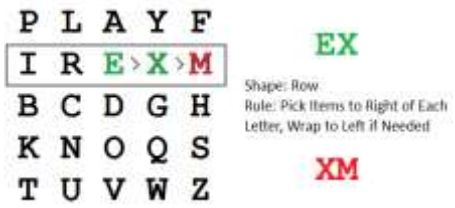
Note: Non-alphabetic characters are preserved using getOrDefault().

Playfair Cipher: Digram Substitution

The Playfair Cipher is a multiple-letter substitution cipher that treats pairs of letters (digrams) as single units. It uses a **5x5 matrix** of letters constructed using a keyword. The matrix is constructed using the keyword, Example, keyword = "**playfairexample**"

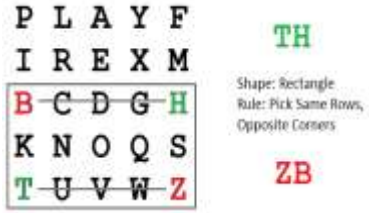
P	L	A	Y	F
I/J	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

Rule 1: Same Row



Rule 2: Same Column

Rule 3: Rectangle



* Note: The letter 'J' is typically omitted or replaced by 'I' to fit the 25-letter grid.

Java Implementation: Playfair Cipher (Part 1)

```java
public class Playfair {

    private final char[][] matrix = new char[5][5];

    public void generateMatrix(String key) {

        String alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ";

        key = key.toUpperCase().replaceAll(regex: "[^A-Z]", replacement: "").replace(target: "J", replacement: "I");

        String combined = key + alphabet;

        StringBuilder unique = new StringBuilder();

        for (char c : combined.toCharArray()) {
            if (unique.indexOf(String.valueOf(c)) == -1) {
                unique.append(c);
            }
        }

        for (int i = 0; i < 25; i++) {
            matrix[i / 5][i % 5] = unique.charAt(i);
        }
    }
}
```

Matrix Generation Logic

01 **Alphabet Setup:** Define a 25-character alphabet, typically omitting 'J' and using 'I' as its substitute.

02 **Normalization:** Convert the keyword to uppercase and append the alphabet to ensure all letters are available.

03 **Deduplication:** Iterate through the combined string, keeping only the first occurrence of each character.

04 **Grid Mapping:** Map the resulting 25 unique characters into a 5x5 two-dimensional array.

Java Implementation: Playfair Cipher (Part 2)

```java
public String encrypt(char a, char b) {

    if (a == 'J') a = 'I';
    if (b == 'J') b = 'I';

    int[] posA = findPosition(a);
    int[] posB = findPosition(b);

    char c1, c2;
    if (posA[0] == posB[0]) {
        // Same row → shift right
        c1 = matrix[posA[0]][(posA[1] + 1) % 5];
        c2 = matrix[posB[0]][(posB[1] + 1) % 5];

    } else if (posA[1] == posB[1]) {
        // Same column → shift down
        c1 = matrix[(posA[0] + 1) % 5][posA[1]];
        c2 = matrix[(posB[0] + 1) % 5][posB[1]];

    } else {
        // Rectangle rule → swap columns
        c1 = matrix[posA[0]][posB[1]];
        c2 = matrix[posB[0]][posA[1]];
    }
    return "" + c1 + c2;
}
```

Pair Encryption Logic

Coordinate Search

The algorithm first scans the 5x5 matrix to locate the row (r) and column (c) indices for both characters in the digram.

Row/Column Wrapping

The modulo operator % 5 is used to implement the "wrap-around" rule, ensuring indices stay within the 0-4 range.

Rectangle Corner Swap

For characters in different rows and columns, the column indices are swapped (c1 ↔ c2) while maintaining the original row indices.

Lab Exercise: Implementation Challenge

The Scenario

You are tasked with securing a communication channel for a small local network. Due to its resistance to simple frequency analysis, you have chosen the **Playfair Cipher** as your primary encryption method.

Your goal is to bridge the gap between theoretical rules and a functional, robust Java implementation.

Technical Tasks

01 Complete the Playfair implementation by adding a method for **plaintext preparation** (handling double letters and odd lengths).

02 Encrypt the message **"Computer Systems"** using the keyword **"balloon"**.

03 Verify your programmatic results against a manual calculation to ensure logic accuracy.

04 Prepare to demonstrate your code and explain how the **matrix rules** are applied during the encryption process.

Lab Exercise:
Solution & Implementation

```java
public String prepareText(String pt) {

    pt = pt.toUpperCase()
            .replaceAll(regex: "[^A-Z]", replacement: "")
            .replace(target: "J", replacement: "I");

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < pt.length(); i++) {
        char first = pt.charAt(i);
        sb.append(first);

        if (i + 1 < pt.length()) {
            char second = pt.charAt(i + 1);

            if (first == second) {
                sb.append(c: 'X');      // insert X between duplicates
            } else {
                sb.append(second);   // normal pair
                i++;                 // skip next char (already used)
            }
        }
    }
    // If length is odd, pad with X
    if (sb.length() % 2 != 0) {
        sb.append(c: 'X');
    }

    return sb.toString();
}
```

Encryption Result

Plaintext

Computer Systems

Prepared Digrams

CO MP UT ER SY ST EM SX

Ciphertext

FB PH QU DS TX TU FK XL

Note: The ciphertext is generated using the 5x5 matrix derived from the keyword "balloon".

Lab Exercise:
Solution & Implementation

```
Run | Debug | Run main | Debug main
public static void main(String[] args) {

Playfair pf = new Playfair();
//Generate Playfair matrix using the key
pf.generateMatrix(key: "balloon");
//Prepare plaintext
String plaintext = "Computer Systems";
String preparedText = pf.prepareText(plaintext);
System.out.println("\nPlaintext     : " + plaintext);
System.out.println("Prepared Text : " + preparedText);
//Encrypt digraphs
StringBuilder ciphertext = new StringBuilder();

for (int i = 0; i < preparedText.length(); i += 2) {
    char a = preparedText.charAt(i);
    char b = preparedText.charAt(i + 1);

    ciphertext.append(pf.encrypt(a, b));
}
//Output result
System.out.println("Ciphertext    : " + ciphertext.toString());
}

}
```

Encryption Result

Plaintext

Computer Systems

Prepared Digrams

CO MP UT ER SY ST EM SX

Ciphertext

FB PH QU DS TX TU FK XL

Note: The ciphertext is generated using the 5x5 matrix derived from the keyword "balloon".

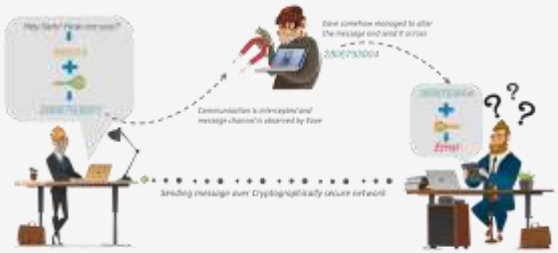
Summary and Next Steps

Lab Recap

Classical ciphers provide the foundation for understanding modern cryptographic principles.

Substitution techniques illustrate the trade-offs between complexity and security.

Practical implementation in Java bridges the gap between theory and application.



Cryptography Ecosystem Overview

Looking Ahead

Next lab will focus on **Transposition Techniques** and more advanced block ciphers.

Ensure your Java environment is set up correctly for the upcoming assignments.